
When Adversarial Data Still Compresses: Towards Robust LLM-Based Compression

Anonymous Author(s)

Affiliation

Address

email

1 Much of the data stored in real-world business systems is text, and lossless LLM-based compressors
2 can outperform traditional schemes such as FSST or Brotli by up to $2\times$ on text data. However,
3 the robustness of LLM-based compression to inputs that induce poor model predictions remains
4 understudied. In this paper, we show that LLM prediction failure does not necessarily imply
5 compression failure: unlikely tokens often represent many source bytes, offsetting their higher
6 coding cost. To study this effect, we decompose LLM-based compression into a two-layer system of
7 tokenization and next-token prediction and evaluate an LLM-based compressor on natural, random,
8 and adversarial inputs. A standard prediction attack, MINPROB, repeatedly selects the model's
9 least-probable next token. Although it increases prediction cost by $5.68\times$ relative to natural text,
10 the resulting output still compresses to 36.2% of the original size. We therefore introduce a new
11 attack objective, MAXSURPRISAL/BYTE, which targets coding cost per source byte rather than per
12 token. This attack expands the encoded output to 144.3% of the original size. Our results show that
13 robustness of LLM-based compression must be assessed end-to-end, accounting for both tokenization
14 and prediction.

15 1 Who Is Actually Doing the Heavy Lifting in LLM-Based Compression?

16 Strings are a core data type in production databases: in an Amazon Redshift fleet study, variable-
17 length strings account for 52.1% of stored columns and 53.8% of predicate columns [1]. Strings are
18 typically user-supplied data that a system has no control over. A malicious actor could construct
19 strings that expand on-disk storage significantly, leading to higher storage costs and unpredictable
20 system performance. Even data that is not intentionally malicious can be a problem: production data
21 is heterogeneous and may not match an LLM's training distribution, leading again to poor next-token
22 predictions and thus poor compression ratios. As data accumulates, compression becomes especially
23 important for cold storage, where space savings can be weighed favorably against access latency
24 and encoding cost [2]. Yet favorable storage economics are not enough: a practical LLM codec
25 must also meet broader systems requirements. Such a codec must preserve queryability, for example,
26 supporting equality checks or joins without full decoding, while remaining robust to *arbitrary* inputs
27 and degrading gracefully when model predictions are poor.

28 To this end, we study a layered architecture designed around the separation between tokenization
29 and predictive compression. The tokenizer first maps strings to stable token IDs that act as a global
30 dictionary and support operations over encoded data without model inference [3]; these IDs form
31 the lossless, queryable representation. For cold storage or transport, an LLM predicts the token IDs
32 and an arithmetic coder [4] compresses them. Because tokenization already constitutes a lossless
33 pre-compression, this predictive layer is optional. When used, arithmetic coding assigns shorter
34 descriptions to more probable tokens [4, 5].

35 Prior LLM codecs do already combine next-token probabilities with arithmetic coding or other
36 lossless coding schemes [6]. Prediction, however, is only one source of compression. Tokenization
37 itself can also act as lossless pre-compression [7]. Byte-Pair Encoding [8], a popular tokenization
38 strategy, originated as a compression algorithm [9], and its effectiveness has been linked to shorter
39 token sequences [10]. From a systems perspective, however, these gains ultimately only matter insofar
40 as the complete codec improves on conventional alternatives such as Brotli [11] and FSST [12].

What remains unclear is how much of an LLM codec’s behavior comes from tokenization versus prediction. Delétang et al. [7] compare ASCII and BPE on natural text, but do not isolate these effects under inputs targeting realized end-to-end size. This distinction matters because prediction cost is measured in bits/token, whereas compression depends on bits/source byte: a poor probability model can still compress when tokens represent many bytes. This fact is neglected in the widely known MINPROB attack that constructs adversarial inputs based on the sequence of lowest-probability tokens. We therefore isolate tokenization and prediction under adversarial inputs to determine when prediction failure becomes compression failure and when tokenization absorbs the cost.

Contributions. We make two contributions: (1) We factor end-to-end compression into tokenizer fertility and predictive coding under natural, random, and adversarial inputs. This decomposition reveals that prediction failure need not imply compression failure: on the `text8` dataset, Qwen2.5-0.5B with arithmetic coding produces a complete stream at 17.6% of raw size, and even when the MINPROB adversary raises prediction cost by $5.68\times$, the stream remains compressive at 36.2%. (2) We formalize token- and byte-level adversarial objectives with realized-code accounting, showing when tokenization absorbs predictive surprise and when that protection against size expansion breaks down. Directly maximizing decoded-byte-normalized surprisal with our MAXSURPRISAL/BYTE attack removes this buffer and expands the complete stream to 144.3% of raw size.

2 Compression Objectives and How to Construct Adversarial Inputs.

A token can be extremely unlikely under the language model while decoding to few source bytes. We formalize the central distinction between prediction difficulty and end-to-end compression failure. We use one notation throughout: let $x = (x_1, \dots, x_N)$ be the generated token sequence, G the candidate token alphabet, $h_t = x_{<t}$ the sequence’s token history, $D = \text{decode}$ denote the decoder, and $p_\theta(v \mid h)$ be the language model probability of token v after history h . Let $|\text{decode}(x)|_{\text{UTF8}}$ denote the number of source bytes obtained after decoding the token sequence x . For example, if $\text{decode}(x) = \text{hello}$, then $|\text{decode}(x)|_{\text{UTF8}} = 5$ bytes. Let $B(x)$ be shorthand for the decoded source size in bytes: $B(x) = |\text{decode}(x)|_{\text{UTF8}}$.

The token-level objective, MINPROB, selects the least-probable admissible continuation $v_{\text{tok}}^* \in G(h)$:

$$v_{\text{tok}}^* := \arg \min_{v \in G(h)} p_\theta(v \mid h) = \arg \max_{v \in G(h)} \{-\log_2 p_\theta(v \mid h)\}. \quad (1)$$

This is the LLM-centric token-level adversarial objective, which maximizes model surprisal and ideal coded bits/token equivalently, but not necessarily bits/source byte. To account for variable-length tokenization, define the incremental decoded byte length contributed by the token v ,

$$\Delta b(v; h) := |\text{decode}(h \parallel v)|_{\text{UTF8}} - |\text{decode}(h)|_{\text{UTF8}}. \quad (2)$$

The byte-aware MAXSURPRISAL/BYTE objective instead selects v_{byte}^* :

$$v_{\text{byte}}^* := \arg \max_{v \in G} \frac{-\log_2 p_\theta(v \mid h)}{\Delta b(v; h)} \quad [\text{bits/byte}]. \quad (3)$$

Unlike Eq. (1), it directly targets the ideal entropy-coding cost in bits/newly decoded source byte. For a complete sequence x , the cumulative ideal model cost is the sequence Negative Log-Likelihood (NLL) $L_\theta(x)$,

$$L_\theta(x) = \sum_{t=1}^N \ell_\theta(x_t \mid x_{<t}) = - \sum_{t=1}^N \log_2 p_\theta(x_t \mid x_{<t}) \quad [\text{bits}], \quad (4)$$

where any fixed initial context is implicit in $x_{<1}$. For serialized output size $S_{\text{codec}}(x) = |\text{SerializeCodec}(x)|$ in bytes, the ideal and realized source-normalized size ratios are

$$R_{\text{ideal}}(x) = 100 \frac{L_\theta(x)}{8B(x)} [\%], \quad R_{\text{real}}(x) = 100 \frac{S_{\text{codec}}(x)}{B(x)} [\%]. \quad (5)$$

Values below 100% denote compression; R_{real} additionally captures arithmetic-coder, metadata, and framing effects.

79 2.1 Prediction Loss Is Not Compression Loss

80 We define tokenizer fertility as the average decoded source-byte count:

$$f(x) = \frac{|\text{decode}(x)|_{\text{UTF8}}}{N} \quad [\text{bytes/token}]. \quad (6)$$

81 Using the sequence NLL $L_\theta(x)$, the ideal predictive rate is

$$R_{\text{ideal}}(x) = 100 \frac{L_\theta(x)}{8|\text{decode}(x)|_{\text{UTF8}}} = \frac{100}{8} \underbrace{\frac{L_\theta(x)}{N}}_{\text{bits/token}} \underbrace{\frac{N}{|\text{decode}(x)|_{\text{UTF8}}}}_{\text{tokens/byte}}. \quad (7)$$

82 Thus high bits/token can be offset by high bytes/token. Tokenization can buffer severe prediction
 83 failure when adversarial tokens represent many source bytes. Looking at Eq. (7), hence, there is
 84 a compression boundary for R_{ideal} that occurs at $\frac{L_\theta(x)}{N} = 8 \frac{|\text{decode}(x)|_{\text{UTF8}}}{N} = 8f(x)$. A token
 85 representing f source bytes can tolerate up to $8f$ bits of surprisal before the predictive layer expands
 86 the source.

87 2.2 Token- and Byte-Level Attacks

88 **Threat Model.** We assume white-box access to the compression pipeline: the adversary can inspect
 89 the tokenizer and model logits and construct length- N adversarial inputs by autoregressive search
 90 over a candidate alphabet G . To ensure that generated token sequences survive decoding and
 91 re-tokenization, every prefix must satisfy $T(\text{decode}(x_{1:t})) = x_{1:t}$; see Appendix 5.2.

92 **Adversarial Input Generation.** We benchmark natural Wikipedia text and random strings against
 93 two adversarial generation policies: $\text{MINPROB}(v_{\text{tok}}^*)$ greedily maximizes Eq. (1), while MAXSUR-
 94 $\text{PRISAL/BYTE}(v_{\text{byte}}^*)$ greedily maximizes Eq. (3). In addition, we provide a *one-byte UTF-8 ablation*
 95 restricting the candidate alphabet G to canonical tokens representing exactly one decoded byte,
 96 thereby removing variation in token fertility. Under this constraint, the multi-byte buffering is
 97 removed and the v_{tok}^* and v_{byte}^* objectives induce the same greedy ranking. These inputs model
 98 adverse blocks arising from either benign data-model misalignment or malicious users. Among the
 99 predictive objectives, $\text{MAXSURPRISAL/BYTE}(v_{\text{byte}}^*)$ directly targets end-to-end size expansion,
 100 whereas token-level loss alone does not determine serialized size.

101 3 Evaluation

102 3.1 Experimental Setup

103 We evaluate Qwen2.5-0.5B [13] on natural text, random text input, full-vocabulary adversarial
 104 sequences, and a one-byte UTF-8 ablation. Each input in the attack evaluation contains 10,000
 105 tokens, except for the full-vocabulary $\text{MAXSURPRISAL/BYTE}(v_{\text{byte}}^*)$ evaluation, which contains
 106 1,024 tokens due to computational constraints.¹ All attacks use the same model, context, and
 107 arithmetic-coding configuration, and generated prefixes must round-trip through tokenizer decoding
 108 and re-encoding.

109 3.2 Results

110 **Natural-to-Random Shift Removes the Predictive Advantage.** On natural text, Qwen+AC achieves
 111 the best compression ratio at 17.6%. Notably, tokenization alone is already a strong baseline,
 112 outperforming FSST, reflecting that Qwen’s learned tokenizer is itself adapted to natural text. The
 113 switch to random input makes this advantage largely disappear: the Qwen token-ID representation
 114 expands to 172.1% of the source size, and although prediction recovers much of this inefficiency,
 115 reducing Qwen+AC to 97.4%, it no longer outperforms Brotli. Nevertheless, random input alone is
 116 insufficient to make Qwen+AC expand beyond the original source size.

¹This shorter evaluation does not change the qualitative conclusion; see Appendix 5.4.

Table 1: **Source-normalized predictive coding under adverse inputs.** Token-level surprisal alone is not predictive of compression failure. ↓ lower is better, ↑ higher is better for compression.

Category	Input / condition	Pred. cost ↓ [bits/token]	Fertility ↑ [bytes/token]	Ideal rate ↓ [bits/byte]	AC rate ↓ [bits/byte]
Baselines	Natural text (text8)	5.11	5.59	0.92	1.13
	Random canonical	8.78	1.31	6.71	7.58
Adversaries	MINPROB (v_{tok}^*)	29.04	7.09	4.10	2.53 [‡]
	MAXSURPRISAL/BYTE (v_{byte}^*) [†]	15.52	1.47	10.53	9.79
Ablation	One-byte UTF-8	8.33	1.00	8.33	9.57

[‡] The unexpectedly low realized AC rate is caused by the inability of a practical coder to represent the model’s extremely small probabilities exactly, coder-support restriction, and induced re-normalization due to padded output positions. An ablation on this effect can be found in Appendix 5.5. Lossless decoding is unaffected. [†] Final 1,024-token evaluation (single run).

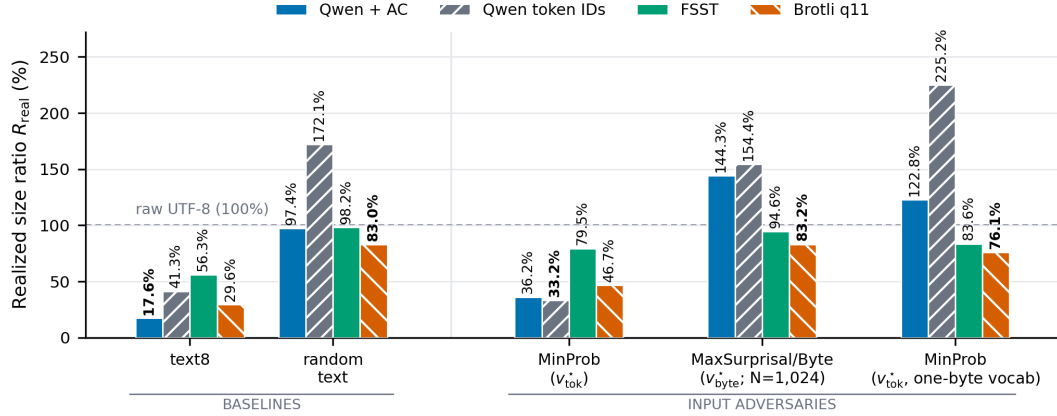


Figure 1: **End-to-end compression across natural vs. adversarial inputs.** Realized size ratio R_{real} for Qwen+AC, Qwen token IDs, FSST [12], and Brotli. Bold values mark the smallest stream for each input. All streams are round-trip-verified. Qwen model weights, tokenizer table are excluded.

Prediction Failure Does Not Imply Compression Failure. MINPROB (v_{tok}^*) raises LLM prediction cost by $5.68\times$, yet the stream remains compressive because tokenizer fertility rises to 7.09 source bytes/token. By selecting unlikely continuations in token space, MINPROB tends to choose vocabulary items that decode to long, multi-byte strings, increasing fertility from 5.59 to 7.09 (Table 1). On the same source bytes, Qwen token IDs are slightly smaller than Qwen+AC, indicating that this robustness comes from the multi-byte token representation rather than predictive coding. Indeed, random inputs are harder than MINPROB for the token-ID baseline, highlighting that tokenization provides substantial pre-compression.

Attacking Fertility-Adjusted Cost Causes Expansion. Removing the fertility buffer translates weak LLM predictions into expansion. Restricting MINPROB to one-byte tokens only fixes fertility at 1 byte/token and expands Qwen+AC to 122.8%. MAXSURPRISAL/BYTE (v_{byte}^*) attacks the same fertility buffer directly over the full vocabulary by maximizing surprisal per decoded byte; it likewise expands Qwen+AC (to 144.3%), making it the strongest end-to-end attack we evaluate.

4 Implications and Conclusion

How LLM-Based Compression Survives Model Errors. LLM compression is an end-to-end systems property: tokenization, prediction, coding, and serialization jointly determine final size. Qwen+AC compresses natural text to 17.6% of raw size and remains compressive under a $5.68\times$ increase in prediction cost, but expands once tokenizer fertility is removed or explicitly targeted. Thus, prediction loss alone does not determine compression failure; tokenizer fertility can absorb substantial model error. More broadly, our results suggest that benign text-model misalignment need not cause catastrophic compression failure as long as the input remains efficiently represented by the tokenizer: severe LLM misprediction can be tolerated when each predicted token still covers sufficiently many source bytes.

References

- [1] Alexander van Renen, Dominik Horn, Pascal Pfeil, Kapil Vaidya, Wenjian Dong, Murali Narayanaswamy, Zhengchun Liu, Gaurav Saxena, Andreas Kipf, and Tim Kraska. Why TPC is not enough: An analysis of the Amazon Redshift fleet. *Proceedings of the VLDB Endowment*, 17(11):3694–3706, 2024. doi: 10.14778/3681954.3682031.
- [2] Andreas Kipf, Tobias Schmidt, Ping-Lin Kuo, Skander Krid, Moritz Rengert, Luca Heller, Andreas Zimmerer, Mihail Stoian, Varun Pandey, and Alexander van Renen. Waiting to Decompress: The Economics of LLM-Based Compression. In *16th Conference on Innovative Data Systems Research, CIDR 2026, Chaminade, CA, USA, January 18-21, 2026*. www.cidrdb.org, 2026. URL <https://vldb.org/cidrdb/2026/waiting-to-decompress-the-economics-of-llm-based-compression.html>.
- [3] Tobias Schmidt et al. Accelerating string-heavy queries with LLM token tables. *PVLDB*, 19, 2026.
- [4] Ian H. Witten, Radford M. Neal, and John G. Cleary. Arithmetic coding for data compression. *Commun. ACM*, 30(6):520–540, 1987. doi: 10.1145/214762.214771. URL <https://doi.org/10.1145/214762.214771>.
- [5] Christian Steinruecken. *Lossless Data Compression*. PhD thesis, University of Cambridge, Cambridge, UK, 2015.
- [6] Chandra Shekhara Kaushik Valmeekam et al. LLMZip: Lossless text compression using large language models. *arXiv preprint arXiv:2306.04050*, 2023.
- [7] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, et al. Language modeling is compression. In *International Conference on Learning Representations*, volume 2024, pages 14165–14181, 2024.
- [8] Rico Sennrich et al. Neural machine translation of rare words with subword units. In *ACL*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162.
- [9] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.
- [10] Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In *EMNLP-IJCNLP*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1141.
- [11] Jyrki Alakuijala et al. Brotli compressed data format. RFC 7932, Internet Engineering Task Force, July 2016. URL <https://www.rfc-editor.org/info/rfc7932>.
- [12] Peter Boncz et al. FSST: Fast random access string compression. *Proceedings of the VLDB Endowment*, 13(11):2649–2661, 2020. doi: 10.14778/3407790.3407851.
- [13] Qwen et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL <https://arxiv.org/abs/2412.15115>.

5 Appendix

5.1 Metric Definitions

For a token sequence x of length N , let $B(x) = |\text{decode}(x)|_{\text{UTF8}}$ be its decoded UTF-8 size in bytes, $L_\theta(x)$ its model negative log-likelihood in bits, and $S_{\text{AC}}(x)$ its arithmetic-coded payload in bits. The metrics reported in Table 1 (prediction cost and tokenizer fertility per token, predictive coding cost per source byte) are re-stated here in more detail:

$$\text{Prediction cost} = \frac{L_\theta(x)}{N} \quad [\text{bits/token}], \quad (8)$$

$$\text{Tokenizer fertility} = \frac{B(x)}{N} \quad [\text{bytes/token}], \quad (9)$$

$$\text{Ideal predictive rate} = \frac{L_\theta(x)}{B(x)} \quad [\text{bits/byte}], \quad (10)$$

$$\text{Realized AC rate} = \frac{S_{\text{AC}}(x)}{B(x)} \quad [\text{bits/byte}]. \quad (11)$$

The ideal rate factorizes as $L_\theta(x)/B(x) = [L_\theta(x)/N]/[B(x)/N]$: high prediction cost can therefore be offset when each token represents many source bytes. To compare encoded sizes directly with raw UTF-8, we report

$$R_{\text{AC}}(x) = 100 \frac{S_{\text{AC}}(x)}{8B(x)}, \quad (12)$$

$$R_{\text{real}}(x) = 100 \frac{|\text{SerializeCodec}(x)|}{B(x)}. \quad (13)$$

Here R_{AC} covers only the arithmetic payload, whereas R_{real} covers the complete serialized stream in bytes, including its dictionary, metadata, and framing. Both are percentages of the raw source size: values below 100% denote compression and values above 100% denote expansion.

5.2 Adversarial Input Construction

The adversarial input is built one token at a time. At every step, (1) the model produces scores for the next possible continuation token in the vocabulary, (2) ranks the candidate tokens according to the attack objective (MINPROB, MAXSURPRISAL/BYTE), and (3) tries candidates in that order until it finds one that remains unchanged after decoding and re-tokenization. The accepted token is then appended and the process repeats.

Canonical Inputs. Not every sequence of valid token IDs can be produced from tokenizing its own decoded bytes. For example, suppose the vocabulary contains tokens for a, b, and ab. The token sequence [a, b] decodes to ab, but the tokenizer may encode ab as the single token [ab]. Hence, decoding and re-tokenizing changes the token sequence. Since our attacks select token IDs directly from the model distribution, allowing such sequences would optimize over token histories that cannot occur for the corresponding source input. It is therefore imperative to require every generated partial sequence to be canonical, i.e., $T(\text{decode}(x_{1:t})) = x_{1:t}$. This ensures that final decoding recovers exactly the same token IDs and bytes.

Candidate Pool. We begin with every token ID shared by the tokenizer and model, then remove all IDs listed in `tokenizer.all_special_ids`. This excludes EOS, padding, chat markers, and other tokenizer-declared control tokens. The resulting pool contains 151,643 IDs for Qwen2.5-0.5B. The notation $G(h)$ in the main paper means the members of this pool that have a finite model score and pass the exact round-trip check in Algorithm 3 after prefix h . Admissibility depends only on the current prefix and proposed next token; there is no lookahead ("one-step-greedy").

Evaluation Costs per Step. Neither attack round-trip tests the full candidate set according to Algorithm 3: candidates are ranked first and canonicity is checked only until one is accepted. MaxSurprisal/Byte additionally computes the context-dependent decoded-byte increment for every candidate before ranking and discards non-positive increments, whereas MINPROB has no byte-length

213 constraint. Computing increments from the complete decoded prefix captures tokenizer-boundary
 214 effects, while the round-trip check rejects candidates whose decoding changes the proposed token
 215 IDs.

216 **Speeding up the Round Trip Check with an Exact Fallback.** Algorithm 3 defines the acceptance
 217 criterion used throughout. To avoid re-tokenizing the entire prefix for every candidate, the imple-
 218 mentation uses a faster suffix check whenever it can guarantee that the earlier part of the prefix is
 219 unaffected. For each accepted prefix, it caches the decoded text, byte offsets, token boundaries, and
 220 pre-tokenizer pieces. It then re-tokenizes only a short suffix when several safety conditions hold,
 221 including that the suffix starts at a token boundary, is at most 512 bytes long, begins with ASCII, and
 222 its earlier tokenization remains unchanged after adding the candidate. If these conditions cannot be
 223 verified, the full-prefix check runs.

Algorithm 1: MINPROB (v_{tok}^*)

Input: a non-empty seed prefix h and target length N .

1. Ask the model for the next-token logits.
- 224 2. Remove candidates with non-finite logits. Sort the rest from lowest logit to highest logit; for equal logits, place the lower token ID first.
3. In that order, apply Algorithm 3. Append the first candidate that passes.
4. Repeat until the complete sequence, including the seed, has N tokens.

Output: the completed token sequence h .

Algorithm 2: MAXSURPRISAL/BYTE (v_{byte}^*)

Input: a non-empty seed prefix h and target length N .

1. Ask the model for the next-token logits and compute float32 log probabilities over the full tokenizer vocabulary.
- 225 2. For every finite candidate v , decode the complete prefixes h and $h\|v$ and measure $d_v = |D(h\|v)|_{\text{UTF8}} - |D(h)|_{\text{UTF8}}$. Remove candidates for which $d_v \leq 0$.
3. Rank the remaining candidates by decreasing $-\log_2 p(v \mid h)/d_v$; for equal scores, place the lower token ID first.
4. In that order, apply Algorithm 3. Append the first candidate that passes.
5. Repeat until the complete sequence, including the seed, has N tokens.

Output: the completed token sequence h .

Algorithm 3: Exact round-trip check

Input: the accepted prefix h and proposed token v .

- 226 1. Decode $h\|v$ with special-token skipping and tokenizer cleanup disabled.
2. Retokenize the decoded text with `add_special_tokens=False`.
3. Accept v exactly when the resulting IDs equal $h\|v$; otherwise reject it and test the next ranked candidate.

Output: accept or reject.

227 5.3 Experimental Setting and Evaluation Protocol

228 **Model and Coder.** We chose Qwen2.5-0.5B because its small size makes full-vocabulary adversarial
 229 scoring tractable while retaining a large vocabulary, and allows the experiments to run locally on a
 230 CPU without requiring GPU access. We use 151,643 non-special candidates, a 1,000-token context
 231 with 100 tokens retained between coding blocks, and KV caching. Arithmetic coding uses a 32-bit
 232 state and frequency total 262,144. The version-1 Qwen+AC stream records the seed, model and coder
 233 configuration, token and raw-byte counts, arithmetic payload, any required token dictionary, and
 234 framing. Model weights and the shared tokenizer table are shared decoder state and excluded from
 235 both Qwen representations.

236 **Inputs and Aggregation.** The primary evaluation uses 10,000-token sequences for natural text,
 237 random canonical input, MINPROB (v_{tok}^*), and the one-byte UTF-8 ablation. MAXSURPRISAL/BYTE
 238 (v_{byte}^*) is reported at the largest feasible budget of 1,024 tokens and is the final evaluation for that
 239 attack. The reported values for the other adversarial conditions are means over three independent
 240 canonical seed runs; the MAXSURPRISAL/BYTE (v_{byte}^*) result is a single run.

241 **Round-Trip Validation and Baselines.** All generated prefixes satisfy

$$T(D(x_{1:t})) = x_{1:t}, \quad t = 1, \dots, N,$$

242 and every complete stream is independently decoded back to identical token IDs or UTF-8 bytes,
 243 as applicable. FSST is evaluated using the authors’ reference implementation [12]; Brotli uses
 244 quality 11; and the Qwen token IDs baseline stores each ID in a fixed-width 18-bit representation
 245 with a stream header.

246 **Extended Experimental Settings.** Table 2 summarizes the remaining generation and evaluation
 settings needed for reproduction.

Table 2: **Extended Experimental Settings.**

Setting	Value used in the reported experiments
Seed prefixes	MINPROB: The = [785], A = [32], and In = [641]. MAXSURPRISAL/BYTE: A = [32]. Total length includes the seed.
One-byte ablation	Scan the vocabulary once; retain v exactly when $[v]$ decodes to one ASCII code point with value 0–127 and that character retokenizes to $[v]$. Qwen provides exactly 128 such IDs.
Model scores	Pretrained model without weight quantization, loaded with the framework’s automatic dtype. Logits are converted to float32 for ranking and logsoftmax. Arithmetic-coder frequency quantization happens only after generation and cannot affect the selected tokens.
Context	KV caching is enabled. At 1,000 context tokens, retain the latest 100 and rebuild the cache. Byte measurement and round-trip validation still use the complete generated prefix.
Length and cost	MINPROB: 10,000 tokens for each of three seeds. MAXSURPRISAL/BYTE: 1,024 tokens for one seed because it performs 151,643 context-sensitive decodes per step.

247

5.4 Additional Results and Accounting

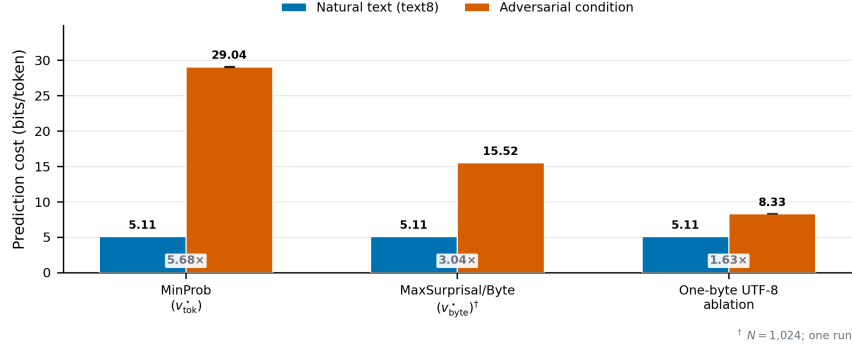


Figure 2: **Adversarial prediction difficulty.** Each condition is compared with the same natural-text prediction cost of 5.11 bits/token. Labels inside the plot report the ratio to text8. Error bars show sample standard deviation across three canonical seeds where available; † marks the final 1,024-token MAXSURPRISAL/BYTE (v_{byte}^*) single run.

Effect of the Shorter MAXSURPRISAL/BYTE (v_{byte}^*) Sequence.

The shorter sequence affects the magnitude but not the qualitative conclusion. Under the same replay configuration, the arithmetic-payload ratio is 136.4%, 133.2%, 131.7%, and 122.4% at $N = 32, 64, 128$, and 1,024, respectively; even the longest evaluated prefix remains larger than raw UTF-8. The complete 1,024-token Qwen+AC stream is 144.3% of its 1,508 source bytes. Moreover, all bars in the MaxSurprisal/Byte group use those same source bytes, so their comparison is size matched. Fixed codec overhead is relatively larger on short inputs, which disadvantages rather than favors the conventional codecs.

Tokenizer Fertility. Table 3 provides a natural-text reference point for the fertility. Note that NLP convention often reports tokens per word. Her fertility is defined as bytes per token: a larger value means that each model prediction covers more source bytes and can therefore absorb more surprisal before the encoded stream expands. Fertility depends on both the tokenizer and the input, which is why adversarial token sequences can differ substantially from the text8 values reported in Section 3.

Table 3: **Tokenizer fertility on the held-out 5 MB text8 test region (855,233 whitespace-delimited words).** The text8 BPE is trained only on the first 90 MB with a 32k vocabulary. Fertility is tokens/word (lower is better); bytes/token is the compression-oriented inverse (higher is better). All tokenizers reconstruct the input exactly. The domain-trained BPE produces 4.4% fewer tokens than Qwen.

Tokenizer	Vocab.	Used	Tokens	Tok./word ↓	Bytes/tok. ↑
ASCII byte	128	27	5,000,000	5.8464	1.0000
text8 BPE (32k)	32,000	26,617	933,499	1.0915	5.3562
Qwen2.5	151,665	25,798	976,828	1.1422	5.1186

Stream Accounting. To separate arithmetic-coding cost from serialization overhead, Table 4 reports the AC payload alongside the complete Qwen+AC stream. We decompose the serialized size as

$$S_{\text{total}} = S_{\text{payload}} + S_{\text{dict}} + S_{\text{frame}},$$

where S_{frame} includes the stream metadata required for decoding. The realized AC rate can also fall below the model NLL because the coder restricts and re-normalizes the model distribution to its supported token IDs and represents the resulting probabilities with finite integer frequencies. For MINPROB, this quantization clips many extremely small probabilities; Section 5.5 isolates these effects.

Table 4: **Realized size ratio by input and codec.** Values are encoded bytes divided by raw UTF-8 bytes and reported as percentages; lower is better. AC payload excludes the Qwen container and is shown only for cost attribution. All entries in the four codec columns are complete, round-trip-verified streams, and bold marks the smallest complete stream per row.

Input	AC payload	Qwen+AC	Qwen token IDs	FSST	Brotli
text8	14.3%	17.6%	41.3%	56.3%	29.6%
Random printable	94.8%	97.4%	172.1%	98.2%	83.0%
MinProb (v_{tok}^*) adversary	32.9%	36.2%	33.2%	79.5%	46.7%
MaxSurprisal/Byte (v_{byte}^* ; $N = 1,024$)	122.4%	144.3%	154.4%	94.6%	83.2%
One-byte UTF-8 ablation	119.5%	122.8%	225.2%	83.6%	76.1%

5.5 Arithmetic-Coder Quantization Sensitivity

A practical real-world observation is that the coder cannot consume arbitrarily small probabilities. For a coding alphabet of size K with frequency total T , it constructs

$$f_i = 1 + \lfloor (T - K)p_i \rfloor + r_i, \quad q_i = f_i/T, \quad (14)$$

where $r_i \in \{0, 1\}$ distributes the counts left after flooring in descending model-probability order. The leading one makes every token decodable, but also imposes $T \geq K$ and a probability floor $p_{\min} = 1/T$.

Thus 2^{14} would be infeasible for the full $K = 151,665$ coder alphabet. The coder alphabet includes the 22 tokenizer IDs excluded from attack selection, hence 151,665 rather than 151,643.

Our reported setting, $T = 2^{18}$, leaves $T - K = 110,479$ counts to represent differences after reserving one count per token. We define a selected token as *clipped* when it receives only that mandatory count.

To test the effect quantization has in practice, Table 5 replays the fixed adversarial token sequences at frequency totals $T \in \{2^{18}, 2^{22}, 2^{26}, 2^{30}\}$; 2^{30} is the largest value that the 32-bit coder implementation can handle. The model, token sequences, decoded bytes, context policy, and arithmetic-coder state size are all held fixed.

The lossless stream can emit only the K tokenizer IDs, so the coder first restricts to that support and re-normalizes. For MINPROB v_{tok}^* this support restriction reduces the ideal rate from 4.10 to 3.79 bits/byte; integer-frequency quantization then reduces it to 2.534 bits/byte at $T = 2^{18}$. This decomposition prevents the whole model-ideal-AC gap from being attributed to integer rounding.

For MINPROB at $T = 2^{18}$, 99.0% of selected tokens are clipped. Raising T monotonically toward the support-restricted ideal shows the following: at the maximum $T = 2^{30}$, clipping falls to 13.8% and realized AC rises to 3.735 bits/byte (46.7% of raw UTF-8), compared with 2.534 bits/byte (31.7%) at 2^{18} . Quantization therefore affects the numerical MINPROB rate, but does not overturn the tokenization result: even the maximum-precision payload remains less than half of raw size because MINPROB has high fertility. For MAXSURPRISAL/BYTE, realized AC changes from 9.790 to 10.481 bits/byte (122.4% to 131.0% of raw) and remains decisively expansive.

Across every row, realized AC differs from quantized ideal by at most 0.002 bits/byte. The reported surprisal error is $\Delta s = -\log_2 q_i + \log_2 p_i$ after restricting p_i to coder support; Table 5 reports its per-token mean and maximum absolute value.

Table 5: **Arithmetic-coder quantization sensitivity.** Fixed token sequences are replayed while only the integer frequency total T changes. Model ideal uses the model’s full output support; support ideal renormalizes over the $K = 151,665$ token IDs available to the lossless coder. Quantized ideal uses the exact integer frequencies. MINPROB values are means over three seeds; MAXSURPRISAL/BYTE is the final $N = 1,024$ run. A token is clipped when its selected frequency is the mandatory count one. $\Delta s = s_{\text{quant}} - s_{\text{support}}$; the final column reports its mean and maximum absolute value.

Condition	T	$p_{\min}$	bits/source byte				Clipped tokens	bits/token
			Model ideal	Support ideal	Quant. ideal	Realized AC		$\overline{\Delta s} / \max \Delta s $
MinProb	2^{18}	3.81×10^{-6}	4.10	3.79	2.534	2.534	99.0%	-8.90 / 29.47
	2^{22}	2.38×10^{-7}	4.10	3.79	3.083	3.083	95.1%	-5.01 / 25.47
	2^{26}	1.49×10^{-8}	4.10	3.79	3.539	3.539	62.8%	-1.77 / 21.47
	2^{30}	9.31×10^{-10}	4.10	3.79	3.734	3.735	13.8%	-0.39 / 17.47
MaxSurprisal/ Byte	2^{18}	3.81×10^{-6}	10.53	10.53	9.792	9.790	39.4%	-1.09 / 16.38
	2^{22}	2.38×10^{-7}	10.53	10.53	10.138	10.138	19.1%	-0.58 / 12.38
	2^{26}	1.49×10^{-8}	10.53	10.53	10.380	10.379	4.2%	-0.22 / 8.38
	2^{30}	9.31×10^{-10}	10.53	10.53	10.482	10.481	2.8%	-0.07 / 4.38

5.6 Reproducibility and Code Availability

The code package to reproduce the experiments contains the implementation, a minimal CPU-only environment, and an experiment entry point.

Environment and Hardware. The reported environment uses Python 3.12.3; the complete dependency graph and package hashes are recorded in `uv.lock`. Key installed versions are PyTorch 2.10.0, Transformers 5.3.0, tokenizers 0.22.2, NumPy 2.4.3, and Brotli 1.2.0. Experiments ran CPU-only under Ubuntu 24.04.1 on one Intel Xeon Gold 5318Y processor (24 physical cores, 48 hardware threads) with 128 GiB RAM.

Model and Data Availability. We use Qwen/Qwen2.5-0.5B and the authors’ reference implementation of FSST, with all artifacts pinned to exact revisions.²

Cross-Hardware Reproduction. We independently reproduced the reported results on two different hardware platforms: a CPU-only server and a high-performance GPU cluster using NVIDIA H100 accelerators under Slurm. For the completed comparable conditions, most experiments reproduced exactly at the precision reported in Table 1: natural text (text8), the random-canonical control, and the one-byte UTF-8 ablation. In the full-vocabulary MINPROB experiment, the largest relative discrepancy across prediction cost, tokenizer fertility, ideal predictive rate, and realized AC rate was 3.0%; the central byte-normalized ideal rate differed by only 0.1%. These small differences are consistent with hardware-dependent floating-point execution.

²Qwen revision: 060db6499f32faf8b98477b0a26969ef7d8b9987;
FSST commit: e638d4cf8c26129d73c242a4127b42b975de5b63;
text8 SHA-256: 6e890197040d37d85bbeb962ae1f041ff1d9a9ca8d20c7d99c85027eebf51dca7.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 3 state the evaluated model, inputs, threat model, and realized-stream results; the introduction limits the contributions to this very specific compression setting.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 4 and Appendix 5.3, 5.5 discuss the single-model scope, finite attack budgets, computational constraints, and observed cross-hardware floating-point variation.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not make any formal proofs; equations are used to define and explain experimental quantities and the attack objectives.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The attack algorithms and evaluation protocol are specified in Appendix 5.2 and Appendix 5.3. Exact environment, hardware, model, commands, and artifact mapping are provided in Appendix 5.6 and the reproduction-package README.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The zipped supplementary provides the experiment implementation, a CPU-only lockfile, pinned model and download instructions, and a single staged runner for the main Qwen attack and tokenizer-fertility results. Third-party model weights and text8 can be downloaded from their public sources.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 3, Appendix 5.2, and Appendix 5.3 specify sequence lengths, seeds, candidate alphabets, context policy, model and coder config.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Sample standard deviations are shown only for the three-seed prediction-cost conditions. MaxSurprisal/Byte is a single run, so the main expansion result has no error bar or significance estimate.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Appendix 5.6 reports the CPU, core count, memory, operating system, and the secondary H100 platform, but per-run wall-clock times and total project compute were not recorded.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses public technical artifacts, introduces no human-subject data or consequential decision system, preserves review anonymity, and releases only experiment code under a permissive license.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: No broader discussion of societal impacts is provided. The paper is strictly focused on the technical aspects.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: There is no new pre-trained model or dataset released as part of the paper. The artifacts created do not pose a risk for misuse.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Qwen2.5-0.5B and FSST are credited and version-pinned, and the supplementary README identifies their Apache-2.0 and MIT licenses and does not redistribute either model weights or text8. However, the upstream text8 download page does not state an explicit dataset license, so this item cannot be answered affirmatively in full.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new experiment-code asset is supplied as an anonymized ZIP with an artifact README documenting scope, setup, exact run commands, hardware, limitations, licensing, external inputs, and the absence of human-subject data. The package also includes an MIT license and a machine-readable result manifest.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work involves neither crowdsourcing nor human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work involves neither crowdsourcing nor human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The language model (Qwen2.5-0.5B) used in this paper is a central component and is used in the pipeline that is under investigation in this paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.